Data_Project — תוכנית עבודה

פרויקט DDR Data Logger על כרטיס Zybo Z7-20

> מצב נוכחי: שלבים 0-7 הושלמו. השלב הבא הוא Stage 8 - AXI DMA + Zynq PS + DDR Hardware Integration.

Stage 0 — GitHub + Project Structure — הושלם

סטטוס: הושלם

מטרה

להקים סביבת עבודה מסודרת ומקצועית לפרויקט, כך שכל הקוד, המסמכים, התמונות, הדוחות וקבצי הכלים יהיו מאורגנים במבנה שמתאים ל-GitHub ולתיק עבודות.

למה השלב חשוב

פרויקט FPGA מקצועי לא כולל רק קוד RTL. הוא צריך לכלול תיעוד, סימולציות, דוחות, תמונות, README ומבנה תיקיות ברור.

תוצר צפוי

Repository מסודר הכולל README, docs, images, rtl, tb, vivado, vitis, python ו-reports.

Stage 1 — Data Protocol — הושלם

סטטוס: הושלם

מטרה

להגדיר את פורמט חבילת הנתונים לפני כתיבת ה-RTL.

למה השלב חשוב

לפני שבונים מערכת שמעבירה נתונים, צריך לדעת בדיוק מה עובר במערכת. פרוטוקול הנתונים מגדיר את מבנה ה-Packet.

תוצר צפוי

קובץ docs/data_protocol.md שמגדיר DATA_WIDTH, Header, Length, Data sequence, Checksum, Expected Packet ו-PASS / FAIL Criteria.

הושלם — Stage 2 — Data Generator + Testbench

סטטוס: הושלם

מטרה

לבנות את בלוק ה-RTL הראשון בפרויקט: Data Generator, ולבדוק אותו באמצעות Testbench וסימולציה ב-Vivado.

למה השלב חשוב

Data Generator משמש כמקור נתונים פנימי ומבוקר, במקום לחבר בשלב מוקדם מקור חיצוני כמו UART, SPI, מצלמה או חיישן.

תוצר צפוי

בלוק Data Generator עובד ומאומת בסימולציה, כולל קוד RTL, קובץ Testbench, תמונת waveform ודוח סימולציה.

הושלם — Stage 3 — Packet Builder FSM + Testbench

סטטוס: הושלם

מטרה

לבנות FSM שמקבלת נתונים מה-Data Generator ומרכיבה מהם Packet מלא לפי הפרוטוקול שהוגדר בשלב 1.

למה השלב חשוב

הבלוק מוסיף מבנה מסודר לנתונים. במקום להעביר רק מספרים גולמיים, הוא יוצר חבילה הכוללת Header, Length, Data ו-Checksum.

תוצר צפוי

rtl/packet_builder_fsm.v, קובץ Testbench וסימולציה שמוכיחה שה-Packet נבנה נכון.

הושלם — Stage 4 — Sync FIFO + Testbench

סטטוס: הושלם

מטרה

לבנות FIFO סינכרוני ולבדוק שמירה על סדר נתונים, כתיבה, קריאה, full, empty, overflow ו-underflow.

למה השלב חשוב

FIFO משמש כחוצץ בין בלוקים. הוא מאפשר לבלוק אחד לכתוב נתונים ולבלוק אחר לקרוא אותם בקצב מתאים בלי לאבד את הסדר.

תוצר צפוי

rtl/sync_fifo.v, קובץ Testbench וסימולציה שמוכיחה שה-FIFO שומר על סדר הנתונים.

הושלם — Stage 5 — AXI-Stream Master + Testbench

סטטוס: הושלם

מטרה

לבנות בלוק שמוציא נתונים בממשק AXI-Stream בסיסי.

למה השלב חשוב

AXI-Stream הוא ממשק נפוץ להעברת נתונים רציפה בין בלוקים ב-FPGA, במיוחד כאשר עובדים עם AXI DMA.

תוצר צפוי

rtl/axis_stream_master.v, קובץ Testbench וסימולציה הכוללת בדיקת backpressure.

הושלם — Stage 6 — Full RTL Integration Simulation

סטטוס: הושלם

מטרה

לחבר את בלוקי ה-RTL המאומתים יחד בסימולציית אינטגרציה מלאה, בלי להסתיר אותם בתוך top סופי אחד.

למה השלב חשוב

השלב מוכיח שכל הבלוקים עובדים יחד, ושומר על הבלוקים נפרדים לקראת אריזת כל אחד מהם כ-Custom IP נפרד ב-Vivado.

תוצר צפוי

tb/tb_data_project_integration.v, סימולציית אינטגרציה מלאה, waveform ו-Console PASS שמראים שה-Packet המלא יוצא דרך AXI-Stream.

הושלם — Stage 7 — Custom IP Packaging + Vivado Block Design

סטטוס: הושלם

מטרה

לארוז כל בלוק RTL כ-Custom IP נפרד, לחבר את הבלוקים בצורה חזותית ב-Vivado Block Design, לבצע Validate, ליצור HDL Wrapper ולהריץ סינתזה.

למה השלב חשוב

שלב זה מעביר את הפרויקט מקבצי RTL נפרדים למערכת IP חזותית ומסונתזת ב-Vivado, שניתן להסביר בצורה ברורה בתיק עבודות ובראיון.

תוצר צפוי

Custom IP נפרד לכל בלוק, Block Design מאומת, HDL Wrapper, תוצאת Synthesis, utilization report, synthesized schematic, commit ו-push.

Stage 8 — AXI DMA + Zynq PS + DDR Hardware Integration

סטטוס: ממתין

מטרה

לחבר את יציאת AXI-Stream של המערכת אל AXI DMA, Zynq PS ו-DDR בתוך Vivado.

למה השלב חשוב

לפני שכותבים תוכנת Vitis, חייבת להיות חומרה מלאה הכוללת Zynq PS, AXI DMA ונתיב DDR. שלב זה יוצר את בסיס החומרה לתוכנה.

תוצר צפוי

Block Design עם Zynq Processing System, AXI DMA, נתיב DDR, bitstream ו-XSA ל-Vitis.

Stage 9 — Vitis C Verification

סטטוס: ממתין

מטרה

לכתוב תוכנת C ב-Vitis שמפעילה את ה-DMA, קוראת נתונים מה-DDR ובודקת שה-Packet תקין.

למה השלב חשוב

ב-Zynq יש שילוב בין PL ל-PS. שלב זה בודק את החיבור בין החומרה שכתבנו לבין התוכנה שרצה על המעבד.

תוצר צפוי

תוכנית C שמאתחלת DMA, קוראת DDR ומבצעת בדיקת Header / Length / Data / Checksum.

Stage 10 — Async FIFO + CDC + Testbench

סטטוס: ממתין

מטרה

להוסיף מעבר נתונים בין שני clock domains באמצעות Async FIFO.

למה השלב חשוב

CDC הוא נושא מרכזי בתעשייה. כאשר שני בלוקים עובדים עם שעונים שונים, אי אפשר להעביר נתונים ישירות בלי סנכרון מתאים.

תוצר צפוי

async_fifo.v וקובץ Testbench עם שני clocks וסימולציה שמוכיחה מעבר נתונים תקין.

Stage 11 — Full Integration with CDC + DMA + DDR

סטטוס: ממתין

מטרה

לשלב את ה-Async FIFO במערכת המלאה עם DMA ו-DDR ולבדוק שהנתונים עדיין נכתבים נכון ל-DDR.

למה השלב חשוב

שלב זה מחבר בין נושאי הליבה של הפרויקט: CDC, FIFO, AXI, DMA ו-DDR במערכת מלאה.

תוצר צפוי

מערכת מלאה עם CDC, DMA ו-DDR, Block Design מעודכן, bitstream ו-XSA מעודכן.

Stage 12 — ILA + Vivado Reports

סטטוס: ממתין

מטרה

להוסיף ILA לדיבוג חומרה אמיתי ולשמור דוחות Vivado.

למה השלב חשוב

בסימולציה רואים מה אמור לקרות. בעזרת ILA אפשר לבדוק מה באמת קורה על הכרטיס, ודוחות timing/utilization חשובים להוכחת איכות המימוש.

תוצר צפוי

screenshots של ILA, timing report, utilization report ותיעוד.

Stage 13 — Python CSV / Graph

סטטוס: ממתין

מטרה

לייצא את הנתונים לקובץ CSV וליצור גרף ב-Python.

למה השלב חשוב

זה מוסיף שכבת הצגה וניתוח נוחה, ומראה יכולת לשלב בין חומרה, תוכנה וכלי ניתוח.

תוצר צפוי

CSV, קוד Python, גרף ותיעוד תוצאה.

Stage 14 — UART or SPI Extension

סטטוס: ממתין

מטרה

להחליף את ה-Data Generator במקור נתונים אמיתי כמו UART או SPI.

למה השלב חשוב

זה הופך את המערכת ממקור נתונים פנימי למערכת שמסוגלת לקבל נתונים מבחוץ.

תוצר צפוי

בלוק תקשורת, בדיקה, וקבלת נתונים אמיתיים במערכת.

Stage 15 — Portfolio Polish

סטטוס: ממתין

מטרה

ללטש את הפרויקט כך שיהיה מוכן להצגה למעסיקים.

למה השלב חשוב

תיק עבודות מקצועי צריך להציג לא רק קוד, אלא גם הסברים, תמונות, waveforms, דוחות, מסקנות ו-Future Work.

תוצר צפוי

GitHub מסודר, README משופר, תמונות, דוחות, הסברים ולקחים מהפרויקט.